

React Custom Hooks Lab: Simple & Practical

MERN Stack Bootcamp – Hands-On Practice

Instructor: Kashif Raza | [LinkedIn](#)

Duration: ~1.5 hours

Difficulty: Beginner



Lab Overview

This lab focuses on **extracting simple, frequently-used patterns** into custom hooks. Each activity is independent. You'll start with inline logic, refactor it into a reusable hook, and verify it works correctly.



Project Setup

```
npm create vite@latest simple-hooks-lab -- --template react
cd simple-hooks-lab
npm install
npm install eslint-plugin-react-hooks --save-dev
npm run dev
```

`.eslinttrc.js`

```
module.exports = {
  plugins: ['react-hooks'],
  rules: {
    'react-hooks/rules-of-hooks': 'error',
    'react-hooks/exhaustive-deps': 'warn'
  }
};
```

◆ Phase 1: useToggle (Boolean State)

Goal: Extract a common open/close pattern used in modals, accordions, and settings panels.

Starter (components/ToggleDemo.jsx)

```
import { useState } from 'react';

export default function ToggleDemo() {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div className="card">
      <button onClick={() => setIsOpen(prev => !prev)}>
        {isOpen ? 'Close' : 'Open'} Panel
      </button>
      <button onClick={() => setIsOpen(true)}>Force Open</button>
      {isOpen && <p>👋 Hello from the panel!</p>}}
    </div>
  );
}
```

Instructions

1. Create hooks/useToggle.js
2. Return an array: [state, toggle, set]
3. Refactor ToggleDemo to use the hook
4. Test: Toggle, force open, verify state consistency

✅ Expected Hook Signature:

```
export function useToggle(initial = false) {
  // [boolean, function, function]
}
```

✅ Success Criteria:

- ☐ Works with true and false initial values
- ☐ toggle() flips state, set() overwrites it
- ☐ No direct useState calls remain in component

◆ Phase 2: usePrevious (Track Past Values)

Goal: Access the previous value of a state or prop without adding extra state variables.

Starter (components/PriceTracker.jsx)

```
import { useState } from 'react';

export default function PriceTracker() {
  const [price, setPrice] = useState(50);
  const [prevPrice, setPrevPrice] = useState(null);

  const updatePrice = (newPrice) => {
    setPrevPrice(price); // ✗ Captures current render's price
    setPrice(newPrice);
  };

  return (
    <div className="card">
      <p>Current: ${price}</p>
      <p>Previous: ${prevPrice ?? 'N/A'}</p>
      <button onClick={() => updatePrice(price + 10)}>+ $10</button>
    </div>
  );
}
```

Instructions

1. Create hooks/usePrevious.js
2. Use useRef to store the value, useEffect to update it after render
3. Return previousValue
4. Refactor PriceTracker to use the hook

✓ Expected Hook Signature:

```
export function usePrevious(value) {
  // returns previous value
}
```

✓ Success Criteria:

- ☐ prevPrice correctly shows the value from the previous render
- ☐ No extra useState for tracking history
- ☐ Ref updates happen synchronously after DOM paint

◆ Phase 3: useWindowSize (DOM Event Listener)

Goal: Safely track browser dimensions for responsive UI without CSS-only media queries.

Starter (components/ResponsiveBox.jsx)

```
import { useState, useEffect } from 'react';

export default function ResponsiveBox() {
  const [width, setWidth] = useState(window.innerWidth);
  const [height, setHeight] = useState(window.innerHeight);

  useEffect(() => {
    const handleResize = () => {
      setWidth(window.innerWidth);
      setHeight(window.innerHeight);
    };
    window.addEventListener('resize', handleResize);
    // ✗ Missing cleanup
  }, []);

  const isMobile = width < 768;
  return (
    <div className={`card ${isMobile ? 'mobile' : 'desktop'}`}>
      {isMobile ? '📱 Mobile Layout' : '💻 Desktop Layout'}
      <p>{width} × {height}px</p>
    </div>
  );
}
```

Instructions

1. Create hooks/useWindowSize.js
2. Add SSR safety: `typeof window !== 'undefined'` check for initial state
3. Add proper cleanup: `window.removeEventListener`
4. Return { width, height, isMobile }
5. Refactor component

✓ Expected Hook Signature:

```
export function useWindowSize() {
  // { width, height, isMobile }
}
```

✓ Success Criteria:

- ☐ Dimensions update instantly on resize
- ☐ No memory leak warnings on unmount
- ☐ Works without crashing in SSR environments
- ☐ isMobile updates correctly at 768px breakpoint

◆ Phase 4: useCopyToClipboard (Web API + Feedback)

Goal: Copy text with success/error state and auto-reset feedback (used in share buttons, code blocks).

Starter (components/CopyDemo.jsx)

```
import { useState } from 'react';

export default function CopyDemo() {
  const [copied, setCopied] = useState(false);
  const [error, setError] = useState(null);
  const text = 'https://example.com/invite?ref=kashif';

  const handleCopy = async () => {
    try {
      await navigator.clipboard.writeText(text);
      setCopied(true);
      setError(null);
      // ❌ Missing auto-reset timeout
    } catch (err) {
      setError('Failed to copy');
      setCopied(false);
    }
  };

  return (
    <div className="card">
      <code className="bg-gray-100 px-2 py-1 rounded">{text}</code>
      <button onClick={handleCopy} className="ml-2">
        {copied ? '✅ Copied!' : '📄 Copy'}
      </button>
      {error && <p className="text-red-500">{error}</p>}
    </div>
  );
}
```

Instructions

1. Create hooks/useCopyToClipboard.js
2. Handle copied, error, and auto-reset after 2 seconds
3. Return [copy, stateObject]
4. Refactor component to use the hook

✅ Expected Hook Signature:

```
export function useCopyToClipboard(resetDelay = 2000) {
  // [copyFunction, { copied, error }]
}
```

✅ Success Criteria:

- ☐ Clipboard API called safely
- ☐ "Copied" resets automatically after 2s
- ☐ Error state clears on next copy attempt
- ☐ Works in modern browsers (falls back gracefully if unsupported)

◆ Phase 5: useCountdown (Timer Management)

Goal: Safely run a countdown without memory leaks or stale intervals.

Starter (components/TimerDemo.jsx)

```
import { useState, useEffect } from 'react';

export default function TimerDemo() {
  const [seconds, setSeconds] = useState(10);
  const [isRunning, setIsRunning] = useState(false);

  useEffect(() => {
    let interval;
    if (isRunning && seconds > 0) {
      interval = setInterval(() => {
        setSeconds(s => s - 1);
      }, 1000);
    }
    return () => clearInterval(interval);
  }, [isRunning, seconds]); // ⚠ Re-creates interval every second!

  return (
    <div className="card">
      <h2>{seconds}s</h2>
      <button onClick={() => setIsRunning(true)}>Start</button>
      <button onClick={() => setIsRunning(false)}>Pause</button>
      <button onClick={() => { setSeconds(10); setIsRunning(false); }}>Reset</button>
    </div>
  );
}
```

Instructions

1. Create hooks/useCountdown.js
2. Use useRef for interval ID to prevent re-creation
3. Expose start, pause, reset methods
4. Return { seconds, isRunning, start, pause, reset }
5. Refactor component

✓ Expected Hook Signature:

```
export function useCountdown(initialSeconds) {
  // { seconds, isRunning, start, pause, reset }
}
```

✓ Success Criteria:

- ☐ Interval runs only once, not recreated on every tick
- ☐ Pause stops timer, resume continues from current value
- ☐ Reset clears interval and restores initial seconds
- ☐ No console warnings about multiple intervals

Troubleshooting Guide

Symptom	Likely Cause	Fix
Hook returns stale value	Missing dependency in <code>useEffect</code> / <code>useCallback</code>	Add variable to deps array
Timer runs multiple times	<code>setInterval</code> recreated on state change	Store ID in <code>useRef</code> , clear before starting
Copy feedback doesn't reset	<code>setTimeout</code> cleanup missing	Return cleanup function or use flag
Window resize crashes SSR	<code>window</code> accessed during render	Initialize state conditionally: <code>typeof window !== 'undefined'</code>
Previous value matches current	<code>useRef</code> updated before render	Move <code>ref.current = value</code> inside <code>useEffect</code>

Debug Pro Tips

```
// 1. Track hook execution
console.log('useToggle called, initial:', initial);

// 2. Verify cleanup
useEffect(() => {
  console.log('🟢 Mounted');
  return () => console.log('🔴 Unmounted');
}, []);

// 3. React DevTools → Components → Hooks tab
// Inspect live state, verify refs, check interval IDs
```



Submission Guidelines



Deliverables

1. **GitHub Repository** containing:
 - `src/hooks/` with all 5 implemented hooks
 - `src/components/` using each hook
 - `README.md` with:
 - Hook signatures & usage examples
 - Screenshot of each working component
 - 1-sentence reflection per hook: "What problem does this solve?"
2. **Clean code:** No ESLint warnings, consistent formatting, JSDoc comments



Evaluation Criteria

Criteria	Points
✓ Hooks follow Rules of Hooks (ESLint clean)	25
✓ Proper cleanup & SSR safety	25
✓ Return values match expected signatures	20
✓ Components refactored correctly	15
✓ Documentation & README	15
Total	100



Bonus (+10)

- Add TypeScript interfaces (`index.d.ts`) for all hooks
- Combine `useToggle` + `useWindowSize` to build a responsive mobile menu
- Deploy to Vercel and share live link

💬 Instructor Note:

Simple hooks are the building blocks of complex systems. When you extract patterns early, your components stay focused on UI, your logic becomes testable, and your team ships faster.

Push your repo, tag me in your PR, and I'll review your hook signatures and cleanup patterns. Keep them focused, predictable, and well-documented.

— Kashif Raza | Small hooks, scalable architecture 🧡